

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 05 -

# MODEL REGISTRY E GOVERNANÇA DE MODELOS

---

## SUMÁRIO

|                                  |    |
|----------------------------------|----|
| O QUE VEM POR AÍ? .....          | 3  |
| HANDS ON .....                   | 4  |
| SAIBA MAIS .....                 | 4  |
| MERCADO, CASE E TENDÊNCIAS ..... | 22 |
| O QUE VOCÊ VIU NESTA AULA? ..... | 25 |
| REFERÊNCIAS .....                | 26 |

EMSE

## O QUE VEM POR AÍ?

Nesta aula, vamos explorar como gerenciar o ciclo de vida de modelos de Machine Learning por meio do uso de Model Registry e de práticas de governança de modelos. Cada vez mais, projetos de ML deixam de ser apenas experimentos isolados para se tornarem soluções críticas em produção, tornando imperativa a capacidade de versionar modelos, controlar seus estágios de implantação, documentar todo o histórico e assegurar conformidade com requisitos técnicos e regulatórios. Você será levado(a) a refletir sobre como organizações líderes conseguem manter centenas de modelos de ML atualizados e confiáveis em ambientes complexos, garantindo que o modelo certo seja usado no lugar e momento certos.

À medida que a área de Machine Learning evolui, entende-se que treinar um modelo de alta acurácia é apenas o começo do desafio. Sem a devida estratégia de governança, empresas podem enfrentar o uso de modelos desatualizados ou incorretos em produção, dificuldade em reproduzir resultados e até riscos de descumprimento de normas legais. É aqui que entram em cena o Model Registry e a governança de modelos: os pilares que conectam a fase de experimentação à fase de deployment, permitindo uma transição segura e auditável para o ambiente de produção. Esta aula vai “quebrar o gelo” em torno deste assunto, mostrando por que ferramentas de registro de modelos (como o MLflow Model Registry) e processos de governança robustos são fundamentais para organizações que desejam escalar suas soluções de IA com confiança e transparência. Prepare-se para descobrir como a teoria se conecta à prática e lembre-se de acompanhar as videoaulas para visualizar esses conceitos em ação e aprofundar seu entendimento!

## HANDS ON

Nesta seção prática, você vivenciará um fluxo profissional completo de registro e promoção de modelos de Machine Learning, utilizando o MLflow Model Registry – uma ferramenta de código aberto amplamente adotada na indústria. A atividade foi concebida para simular como empresas que operam modelos de ML em produção controlam versões de modelos, avaliam sua qualidade e realizam a transição entre diferentes ambientes (staging, production etc.) de forma segura.

Durante este Hands On, você irá registrar modelos a partir de experimentos conduzidos com o MLflow, versionar modelos automaticamente, gerenciar estágios de vida (promovendo modelos de Staging para Production ou arquivando versões antigas) e aplicar práticas de governança, adicionando descrições e tags de aprovação aos seus modelos. Essas etapas refletem um ciclo real de MLOps corporativo: desde a experimentação inicial até a gestão de modelos aprovados para produção, sempre enfatizando rastreabilidade e controle.

Código completo da prática disponível no GitHub:  
<https://github.com/Cataldir/Materiais-MLET/tree/main/fase-02-containers-e-ambientes-reprodutivos/04-dvc-mlflow>.

## SAIBA MAIS

Conforme projetos de Machine Learning avançam do ambiente de laboratório para aplicações reais, surgem desafios complexos na gestão de modelos em produção. Diferentemente de um código tradicional, os modelos de ML incorporam dependências de dados e comportamentos estatísticos dinâmicos, exigindo atenção contínua mesmo após seu desenvolvimento. É comum que uma empresa mantenha múltiplas versões de um mesmo modelo, treinadas com dados e parâmetros diferentes, ao mesmo tempo em diferentes estágios (por exemplo, uma versão em produção e outras sendo testadas). Além disso, há a necessidade de avaliar modelos por diversas métricas (acurácia, desempenho, latency etc.) e de implantar modelos em ambientes variados (desenvolvimento, homologação, produção). Some-se a isso exigências regulatórias e de auditoria – como explicar decisões tomadas por um modelo de IA ou garantir que ele cumpra legislações setoriais – e fica claro que a falta de governança pode gerar riscos significativos para organizações que dependem de ML.

Sem uma governança adequada, os riscos incluem: deploy acidental de modelos incorretos ou desatualizados em sistemas de produção; falta de rastreabilidade (não se sabe dizer qual versão de um modelo originou uma determinada previsão ou resultado de negócio); dificuldade de reverter modelos problemáticos (por não se saber qual modelo estava em produção antes); e não conformidade regulatória, especialmente em setores como financeiro e saúde, onde órgãos reguladores exigem documentação e controle rigoroso dos modelos algorítmicos. De fato, em uma pesquisa da [Algorithmia](#) com 400 empresas sobre obstáculos à adoção de IA, 56% dos entrevistados apontaram requisitos de governança, segurança e auditoria como o maior desafio para colocar modelos de ML em produção. Isso demonstra que, à medida que sistemas de IA se tornam onipresentes nas operações de negócio, a governança de modelos desponta como elemento crítico para o sucesso em Machine Learning.

### O que é model registry?

Para enfrentar esses desafios de controle e rastreabilidade, a comunidade de ML desenvolveu a ideia de Model Registry (Registro de Modelos). Um Model Registry nada mais é que um repositório centralizado e estruturado para armazenar, versionar e gerenciar modelos de Machine Learning ao longo de todo o seu ciclo de vida. Segundo a documentação do MLflow, “o Model Registry do MLflow é um armazenamento central de modelos, com um conjunto de APIs e interface visual, projetado para gerenciar colaborativamente todo o ciclo de vida de um modelo de ML, fornecendo informações de linhagem (qual experimento e execução do MLflow produziu o modelo), versionamento e suporte a metadata, tags e anotações”. Em outras palavras, um Model Registry permite que diferentes membros de uma equipe de ciência de dados/engenharia de ML compartilhem modelos de forma organizada e controlada, evitando o caos de arquivos de modelo dispersos em drives pessoais ou em pastas de projeto.

No contexto do MLflow, que é uma plataforma open-source de gerenciamento de ciclos de vida de ML, o Model Registry é um componente nativo que integra-se aos demais recursos do MLflow. Sempre que um modelo é logado em uma execução do MLflow Tracking, pode-se registrá-lo no Model Registry com um nome único, passando a ser identificado como um modelo registrado. Cada novo registro de um modelo gera automaticamente uma nova versão dentro daquele nome. Assim, se você registrou um modelo chamado ChurnClassifier, a primeira versão será a 1; ao treinar uma melhoria e registrá-la sob o mesmo nome, ela será salva como versão 2, e assim por diante. O Model Registry do MLflow fornece também uma interface gráfica na qual é possível visualizar todos os modelos registrados, seus diagramas de linhagem (vinculação com experimentos e runs), suas versões, métricas, comentários e transições de estágio.

Em suma, Model Registries (seja o do MLflow ou de outras plataformas, como o Model Registry do Amazon SageMaker ou do Azure ML Studio) oferecem um local unificado para acompanhar modelos desde a criação até a descontinuação, fomentando colaboração e repetibilidade. Sem esse tipo de ferramenta, a gestão de modelos em larga escala se torna inviável e sujeita a erros humanos. Modelos de ML

são ativos vivos, que evoluem com novos dados e técnicas; o registro centralizado nos permite acompanhar essa evolução com segurança.

### Estágios do ciclo de vida do modelo

Um ponto central na governança é definir estágios claros no ciclo de vida de cada modelo e controlar as transições entre esses estágios. Em projetos práticos, costuma-se adotar um fluxo de promoção aproximadamente assim:

**Development/None:** fase inicial de desenvolvimento, em que o modelo é treinado e avaliado localmente. Versões registradas podem ficar sem estágio definido (None) até que se decida promover uma candidata.

**Staging (Validação):** modelos candidatos à produção passam por testes rigorosos em um ambiente de staging (homologação). Aqui, avalia-se desempenho em dados não vistos, robustez a diferentes cenários e o cumprimento de critérios de negócio (por exemplo, um modelo de crédito deve atingir um mínimo de acurácia e ao mesmo tempo não apresentar viés indevido contra certos grupos).

**Production:** modelos em uso ativo em sistemas reais, servindo previsões para usuários ou processos de negócio. Geralmente, apenas uma versão de cada modelo problema está no estágio Production por vez (salvo em implementações de A/B testing ou canary releases, discutidas adiante).

**Archived:** modelos retirados de produção ou obsoletos são arquivados. O archive não deleta o modelo (preserva a integridade para auditoria), mas serve para sinalizar que aquela versão não deve mais ser usada ativamente.

A transição entre estágios é sempre intencional e auditável. Ferramentas de Model Registry registram automaticamente quem promoveu ou arquivou um modelo, quando e (opcionalmente) por que. Em sistemas corporativos, é comum implementar fluxos de aprovação: por exemplo, um workflow onde um cientista de dados propõe a promoção de um modelo a Production, um líder técnico revisa as métricas e aprova, e então a promoção ocorre com registro dessa decisão. Essas informações ficam gravadas como parte do histórico do modelo – no MLflow, poderíamos usar as tags ou annotations para indicar “aprovado por Fulano em 09/02/2026”. Assim, o Model

Registry atua como um mecanismo de controle de qualidade, garantindo que apenas modelos devidamente validados cheguem à produção.

### **Governança de modelos e políticas de aprovação**

Chama-se governança de modelos o conjunto de políticas e processos que asseguram a qualidade, conformidade e rastreabilidade de modelos de ML em todo seu ciclo de vida. Diferentemente de um enfoque meramente técnico, a governança engloba preocupações de negócio, risco e compliance. Alguns princípios fundamentais de governança incluem:

A definição de métricas mínimas de qualidade é essencial para garantir que um modelo esteja apto para implantação. Esses critérios funcionam como portões de aprovação que só se abrem quando o modelo atinge valores previamente estabelecidos, como exigir que um classificador de fraude apresente pelo menos 95% de recall e 90% de precisão ou que um modelo de visão computacional supere a acurácia da versão atualmente em produção. Esse processo pode ser automatizado por ferramentas de CI/CD, que verificam se o modelo atende às exigências antes de avançar no fluxo.

A governança também deve assegurar a conformidade regulatória e ética, garantindo que os modelos respeitem legislações como a GDPR ou regulamentos setoriais. Em 2021, a União Europeia propôs o primeiro arcabouço regulatório de IA, estabelecendo níveis de risco e impondo requisitos rigorosos para sistemas classificados como de alto risco, incluindo robustez, documentação, transparência e supervisão humana. Mesmo fora de ambientes regulados, a aderência a políticas internas e diretrizes corporativas de ética em IA continua sendo obrigatória.

A rastreabilidade e a reprodutibilidade dependem de registros completos sobre a criação de cada modelo, desde a origem dos dados e versões de software até hiperparâmetros e métricas de treinamento. Estudos apontam que a reprodutibilidade exige armazenar metadados detalhados e o Model Registry desempenha papel crítico ao vincular cada versão de modelo ao experimento que a originou, facilitando reconstrução e auditorias, especialmente em setores como o financeiro.



O controle de versões e a capacidade de rollback são mecanismos fundamentais para mitigar riscos em produção. Poder reverter rapidamente para uma versão anterior caso a nova apresente problemas é crucial. No MLflow, todas as versões permanecem armazenadas e retornar uma versão mais antiga ao estágio Production pode ser feito em segundos por meio da API.

Apesar do alto grau de automação desejável em MLOps, decisões de promoção de modelos geralmente exigem aprovação humana. Essa camada de governança envolve comitês ou responsáveis técnicos que avaliam métricas, riscos e conformidade antes que um modelo seja oficialmente colocado em produção. Ferramentas de registro permitem documentar aprovações, fortalecendo a confiança e a auditabilidade do processo.

Por fim, a governança continua após o deploy, exigindo monitoramento contínuo do comportamento do modelo em produção. É necessário definir métricas de desempenho como acurácia, latência e uso de recursos, além de estabelecer quais equipes respondem a alertas e como tratar anomalias. Estratégias de realimentação, como pipelines periódicos de retreino em caso de data drift ou degradação de performance, garantem que o modelo permaneça alinhado às condições reais de operação.

Por exemplo, bancos e fintechs implementam programas rígidos de governança de modelos de crédito e detecção de fraudes para cumprir normas de órgãos reguladores (como o Banco Central ou SEC) – essas instituições frequentemente exigem documentação detalhada e validação independente dos modelos antes do deploy. Segundo um relatório de 2021, 35,8% das empresas na Alemanha consideravam requisitos legais (como transparência em decisões algorítmicas) um dos principais desafios para levar AI para produção. Isso reflete como não há mais opção: a governança de modelos tornou-se mandatória, não apenas para evitar sanções, mas também para assegurar qualidade e confiança nas soluções de IA em todos os setores.

### Model registry e reprodutibilidade em MLops

Do ponto de vista técnico, um dos grandes benefícios de se adotar um Model Registry é facilitar a reprodutibilidade e a auditoria. Cada versão de modelo registrada carrega consigo links para o experimento (ou run) do qual se originou, onde estão armazenados os parâmetros de treino, o código do modelo (se usando MLflow Projects ou outro gerenciador de experimentos) e até mesmo uma snapshot do conjunto de dados de treinamento utilizado. Isso significa que, dada uma versão em um Model Registry, consegue-se recriar exatamente o modelo (ou, pelo menos, obter todas as informações necessárias para recriação). Essa capacidade tem dupla utilidade: primeiro, permite investigação de incidentes (por exemplo, entender por que um modelo tomou determinada decisão analisando suas features de treino e hiperparâmetros); segundo, possibilita refazer o treinamento do modelo caso seja necessário atualizar o ambiente ou o stack de software. Em contextos regulados – como no desenvolvimento de algoritmos médicos ou sistemas de crédito – essa reprodutibilidade não é luxo, sim obrigação. Sem um mecanismo de registro, seria virtualmente impossível provar a origem de um modelo específico ou recriar seus resultados para efeitos de auditoria.

Sculley et al. (2015) salientam que a maior parte do esforço em sistemas de ML reside fora do código do modelo em si, abarcando dados, infraestrutura, monitoramento etc. Em famosos termos dos engenheiros do Google, o script de aprendizado é apenas a “pequena caixa preta no meio de um diagrama, rodeada por um grande oceano de código de suporte”. Essa observação, presente em um influente artigo chamado “Hidden Technical Debt in ML Systems”, reforça a ideia de que manter um modelo útil ao longo do tempo exige gerenciar todas as peças em torno dele: dados atualizados, pipelines reproduzíveis, ambiente de execução consistente e controles de acesso. O Model Registry atua justamente nesse “oceano” de componentes de suporte, fornecendo o ponto de ancoragem para integrar diversas funções de MLops. Por exemplo, combinando um registry com um Feature Store, garante-se que as features usadas no treinamento de um modelo fiquem documentadas e versionadas junto dele; integrando com sistemas de monitoramento,

pode-se rotular versões do modelo com indicadores de performance (ex.: tags de métricas de negócio) conforme elas rodam em produção.

### **Model registry no contexto de mlops**

Em um ambiente de MLOps completo, o Model Registry desempenha o papel de ponte entre a fase de experimentação e a fase de produção de modelos. Ele funciona como a “fonte da verdade” para pipelines automatizados de implantação (CI/CD de ML). Por exemplo, imagine um pipeline de entrega contínua configurado para um serviço de IA: quando um novo modelo é registrado e marcado como Production no Registry, uma etapa de CI/CD pode acionar automaticamente o pacote ou containerização desse modelo e implantá-lo em um cluster de inferência. Sem o registro central, seria necessário configurar manualmente qual artefato (por nome de arquivo, por exemplo) é o modelo mais recente – uma prática sujeita a erros.

Além disso, o registry possibilita estratégias avançadas de implantação. Com ele, torna-se viável adotar Blue/Green deployments, Canary releases ou A/B tests, em que múltiplas versões do mesmo modelo ficam ativas simultaneamente para diferentes parcelas de usuários. O registro de modelos facilita esse processo ao permitir, por exemplo, que versões específicas sejam identificadas por aliases ou tags. O MLflow Model Registry introduziu recentemente o conceito de alias de modelo (a partir da versão 2.x do MLflow), em que um nome alternativo pode ser atribuído a uma versão. Assim, poderíamos referenciar sempre o modelo “ChurnClassifier@produto” para nos referirmos ao modelo de churn atualmente em produção e alterar o alias para apontar para uma nova versão quando esta for promovida. Esse tipo de mecanismo permite que sistemas externos acessem sempre o modelo correto através de um identificador fixo, enquanto internamente a equipe de MLOps pode atualizar o apontamento do alias para a versão aprovada.

Outro aspecto essencial é que o registry serve de base para pipelines de CI/CD específicos de ML. Ferramentas como Apache Airflow, Kubeflow Pipelines ou serviços gerenciados (AWS SageMaker Pipelines, Google Cloud Vertex Pipelines etc.) podem interagir com o registro para tomar decisões automatizadas. Por exemplo, um pipeline de treinamento periódico pode comparar o desempenho de um novo modelo com o

atual modelo em produção (obtendo este último do registry) e, se o novo modelo superar o antigo em métricas definidas, desencadear automaticamente o processo de promoção. Esse conceito de automatização de decisões de promoção está na fronteira das práticas de MLOps: já existem soluções de AutoML Ops que integram avaliação contínua de modelos em produção e seleção automática de modelos ótimos, sempre com salvaguardas de governança para revisão humana quando necessário (Godse; Soddone, 2022 – whitepaper da Microsoft sobre responsible AI).

Por fim, vale mencionar que a adoção de Model Registry e governança tem crescido rapidamente. Empresas de tecnologia e setores tradicionais já reportam ganhos significativos com essas práticas. O Uber, por exemplo, desenvolveu o Michelangelo, sua plataforma de ML interna, que gerencia automaticamente o versionamento e a implantação de milhares de modelos de aprendizagem de máquina em serviços como previsão de ETA e despacho de. Nesse contexto, o registry de modelos provê a base para suportar o escalonamento: no Uber, mais de 100 algoritmos de ML eram promovidos a produção diariamente com controle de versão e monitoramento automatizado, algo impossível de gerir manualmente. Iniciativas similares existem no Facebook (FBLearner Flow), Airbnb (Bighead) e outras grandes empresas, todas convergindo para o mesmo princípio: “tratamento de modelos como produtos de software, com controle de versão, testes e deploy contínuo”.

### **Arquitetura e componentes de um model registry (MLflow)**

Focando no MLflow Model Registry, sua arquitetura ilustra bem os componentes típicos de um sistema de registro de modelos corporativo. O MLflow utiliza um backend de metadados (um banco de dados SQL, como PostgreSQL, MySQL ou SQLite) para armazenar todas as informações de modelos e versões. Quando você registra um modelo, dados como nome do modelo, número da versão, data/hora do registro, estágio atual, autor, tags e artefatos associados são salvos nesse banco de metadados central. Para ambientes de produção com alta concorrência de usuários, recomenda-se um banco robusto (PostgreSQL ou MySQL) para garantir escalabilidade e consistência.

Cada versão de modelo no registry é imutável no que tange ao artefato do modelo: uma vez registrada a versão 1, por exemplo, não se pode alterar qual artefato ela aponta – se precisarmos de mudanças, isso resultaria em uma nova versão (v2). Apenas metadados descritivos podem ser atualizados. Esse versionamento sequencial imutável garante auditabilidade completa: sabe-se exatamente que o modelo v1 permanece tal como foi originalmente aprovado, oferecendo confiança para auditorias ou comparações históricas. O Model Registry do MLflow integra-se fortemente com o componente de Tracking: cada versão armazena referência ao Run ID do experimento que originou o modelo, de modo que seja possível navegar facilmente do modelo registrado para ver quais parâmetros, dados e código o produziram. Essa integração de tracking + registry efetivamente implementa o conceito de linhagem de modelos, facilitando a reprodutibilidade já mencionada.

A interface de gerenciamento do registry no MLflow se dá tanto via APIs (como usamos no Hands On) quanto via uma interface gráfica web. Na UI, é possível comparar visualmente métricas de diferentes versões, promover modelos entre estágios com cliques e avaliar segurança e integridade dos artefatos. Em um contexto corporativo, essa interface pode ser utilizada por product owners, cientistas de dados e DevOps engineers em conjunto, servindo como ponto comum de decisão. O MLflow permite até notificações e webhooks – por exemplo, programar um webhook que acione um alerta no Microsoft Teams ou Slack toda vez que um modelo for promovido para Production. Em soluções de mercado (como o registry integrado ao Databricks ou ao Azure Machine Learning), há recursos adicionais como controle de acesso por usuário (permissões de quem pode registrar ou promover modelos) e integração com catálogos corporativos. Isso demonstra uma tendência: ferramentas de registro de modelos estão evoluindo para servir como centros de governança dentro das plataformas de MLOps, com segurança e compliance desde a raiz.

### **Políticas de promoção e aprovação**

A efetividade de um Model Registry depende não apenas da ferramenta, mas também das políticas organizacionais estabelecidas em torno dele. Promover um modelo não deve ser um ato arbitrário, mas sim o resultado de um processo bem

definido. Boas práticas de governança sugerem implementar gates de qualidade automatizados nos pipelines de CI/CD de modelos. Por exemplo: um modelo candidato só avança para Staging se suas métricas de validação superarem as do modelo atualmente em produção (dentro de uma margem significativa). Esse tipo de gate pode ser implementado como código – por exemplo, um script que compara as métricas do novo modelo com as do modelo de referência obtido via API do registry, aprovando ou reprovando a promoção conforme o caso.

Além dos gates, há checkpoints de aprovação humana. Tipicamente, antes de ir para Production, um modelo requer sinal verde de stakeholders. No MLflow, podemos usar tags como `approved_by` ou `approval_notes`, ou ainda campos de descrição, para registrar a evidência dessa aprovação (por exemplo: “Versão 5 aprovada em 10/02/2026 pelo Comitê de IA após passar em testes de stress e bias”).

Esses registros textuais complementam as métricas automatizadas com contexto de negócio e decisões de design. Vale ressaltar que, em setores altamente regulados (financeiro, saúde etc.), a promoção de modelos sem aprovação formal não é aceitável – faz parte da conformidade que cada mudança significativa passe por revisão e fique documentada. A falta desse controle pode implicar penalidades legais e, ainda mais importante, riscos à qualidade do produto ou serviço caso um modelo problemático seja colocado em uso sem a devida supervisão.

Uma vez definidos os processos de aprovação, o Model Registry facilita também o rollback, conforme já discutido. Imagine que um modelo versão 8 foi promovido, mas apresentou degradação de performance em produção; com poucos comandos ou cliques, podemos retornar o modelo anterior (versão 7) ao estágio Production. Todo o histórico de eventos – a promoção da v8, a detecção do problema, e a repromulgação da v7 – fica armazenado. Essa rastreabilidade é valiosa para análise post-mortem e melhoria contínua do processo de MLOps.

Por fim, convém mencionar que governança eficaz frequentemente envolve testes em produção controlados, como canary releases, em que uma nova versão de modelo é implantada para uma pequena parcela de usuários inicialmente. O Model Registry pode ser usado para gerenciar isso rotulando versões de teste com tags específicas (ex.: `stage:canary` ou associando um alias tipo `candidate`). Serviços de inferência podem então direcionar, digamos, 5% do tráfego para o modelo com tag

canary, mantendo 95% no modelo production. Após coletar dados suficientes e se o modelo candidato se provar superior, ele “toma o lugar” do modelo antigo em Production. Técnicas assim reduzem riscos e fazem parte de uma governança moderna, na qual decisões são suportadas por evidências experimentais em ambiente real.

### Gestão de dependências e ambiente

Outro desafio técnico na operacionalização de modelos é assegurar que o modelo treinado seja executado corretamente no ambiente de produção. Diferenças sutis de ambiente (versões de bibliotecas, configurações de hardware, libs de pré-processamento) podem fazer um modelo funcionar bem em testes, mas falhar em produção. Por isso, ferramentas de Model Registry costumam incluir mecanismos robustos para gerenciar dependências e ambientes de execução.

No MLflow, todo modelo registrado carrega consigo uma especificação de ambiente condizente com sua execução, normalmente via um arquivo `conda.yaml` ou semelhante. Esse arquivo descreve as bibliotecas Python requeridas (com versões) e até dependências de sistema. Assim, ao descarregar um modelo do registry para produção, o MLflow pode recriar o ambiente exato em que ele foi treinado ou pelo menos verificar se o ambiente atual satisfaz os requisitos, mitigando problemas de compatibilidade. Alternativamente, pode-se usar a funcionalidade de empacotamento em contêiner Docker: o MLflow consegue gerar um Docker image a partir de um modelo registrado, já incluindo todas as dependências necessárias. Essa abordagem de containerização garante isolamento completo do modelo, facilitando sua portabilidade entre diferentes ambientes (dev/staging/prod) e até entre nuvens distintas.

Outro ponto crítico é a gestão de artefatos auxiliares do modelo. Muitos modelos de ML dependem de componentes de pré-processamento (por exemplo, escalonadores de variáveis, codificadores de categorias, dicionários de embedding etc.). Um bom Model Registry permite anexar esses artefatos a cada versão de modelo, garantindo que nada “fique para trás”. No MLflow, por exemplo, pode-se salvar um pipeline de pré-processamento juntamente com o modelo propriamente dito,

de modo que ao carregar o modelo em produção esse pipeline também seja recuperado e aplicado. Essa prática reforça a consistência entre treinamento e inferência, outro pilar da confiabilidade. Afinal, de nada adianta termos o melhor modelo se os dados fornecidos a ele em produção não passarem pelas mesmas transformações que foram aplicadas nos dados de treinamento – isso causaria skew e erros previsíveis.

Por fim, recursos de validação de schema complementam a governança técnica. Quando registramos um modelo, podemos (e devemos) especificar o formato dos dados de entrada esperados e da saída produzida (Model Signature no MLflow). Essa informação pode ser usada em tempo de inferência para verificar automaticamente se os dados que um sistema está enviando ao modelo possuem os atributos e tipos corretos. Caso não correspondam ao schema esperado, a chamada de inferência pode ser rejeitada ou redirecionada, prevenindo erros silenciosos. Mais uma vez, essa é uma característica que só se torna viável quando temos um ponto central (o registro de modelos) conhecendo as expectativas de cada versão de modelo em produção.

### **Monitoramento e observabilidade pós-deploy**

Uma vez que um modelo entra em produção, as atividades de governança prosseguem na forma de monitoramento e observação contínuos. Diferente de software convencional, modelos de ML podem perder desempenho ao longo do tempo devido a mudanças nos dados e no mundo real. Portanto, estabelecer um sistema de monitoramento eficaz faz parte da governança.

Em produção, é essencial acompanhar continuamente um conjunto de métricas que revelam o comportamento operacional e a qualidade do modelo implantado. As métricas de serviço incluem a latência de inferência, que representa o tempo de resposta do modelo, bem como a taxa de erro e o consumo de CPU, GPU e memória. Embora sejam métricas comuns em sistemas de software tradicionais, no contexto de Machine Learning elas exigem correlação direta com a versão específica do modelo utilizada em cada requisição. Por isso, integrações entre o Model Registry e ferramentas de observabilidade tornam-se fundamentais, permitindo que o



identificador da versão seja automaticamente anexado a logs e métricas, facilitando análises de impacto quando, por exemplo, há aumento de latência decorrente de uma versão particular do modelo.

Além disso, quando o tipo de aplicação permite retorno de feedback, torna-se possível mensurar continuamente métricas de acurácia após a implantação, como precisão, revocação e AUC. Esse monitoramento comparativo entre o desempenho atual e o desempenho observado no treinamento permite detectar degradações estatisticamente significativas ao longo do tempo. Ao identificar essa deterioração, mecanismos de governança podem acionar investigações ou mesmo iniciar de forma automática um processo de retreinamento, fechando o ciclo operacional e evitando pressupor que o modelo manterá desempenho ideal indefinidamente. Ferramentas como Evidently AI e WhyLabs têm surgido justamente para identificar fenômenos como data drift e concept drift e podem se integrar ao registro de modelos para associar relatórios de drift às versões correspondentes.

Outro componente essencial da governança é o registro estruturado de logs de predições. Cada predição emitida por um modelo considerado crítico deve ser registrada com informações como timestamp, versão do modelo, dados de entrada e resultado produzido. Embora isso requeira maior capacidade de armazenamento, o valor para fins de auditoria e investigação é significativo. Em situações regulatórias sensíveis, como modelos de concessão de crédito, essa rastreabilidade permite identificar quais decisões foram tomadas por quais versões, bem como os dados e suposições que as fundamentaram. Esses registros também possibilitam explicar e reproduzir decisões individuais. Caso um cliente questione, por exemplo, a negativa de um empréstimo, a organização pode recuperar exatamente a predição efetuada, seus dados de entrada e a versão do modelo utilizada, gerando explicações com técnicas de interpretabilidade, como SHAP ou LIME. Essa capacidade fortalece a transparência e a confiança no uso de IA em ambientes críticos.

Como parte da governança, devem ser estabelecidas políticas de retenção desses logs e métricas, de forma a equilibrar compliance e custo. Regulamentos como a GDPR podem exigir que informações pessoais usadas em modelo sejam deletadas após certo período, o que entra em conflito com o desejo de guardar logs para auditoria. Uma solução comum é anonimizar ou agregar informações nesses logs, de

modo que possam ser mantidos por períodos mais longos sem violar privacidade – essa é uma decisão de governança que envolve times de dados, jurídico e segurança da informação.

### **Governança em ambientes regulados**

Setores como o financeiro, saúde e governo apresentam requisitos de governança especialmente rigorosos para modelos de IA, visando gerenciar riscos éticos e de compliance. Nesses contextos, um Model Registry bem estruturado ajuda a atender diversas exigências normativas. Por exemplo, bancos nos EUA seguem diretrizes do OCC (Office of the Comptroller of the Currency) para gerenciamento de riscos de modelos, que recomendam manter inventários de todos os modelos, validações independentes e auditorias periódicas dos mesmos. Um registro de modelos viabiliza o inventário centralizado, onde cada modelo em uso (seja um algoritmo de crédito, detecção de fraude ou precificação de ativos) está documentado com suas suposições, autores, versões e registros de validação.

Práticas de governança em ambientes regulados exigem um nível elevado de explicabilidade e documentação detalhada dos modelos. Sistemas de IA utilizados em saúde ou finanças precisam ser totalmente explicáveis, o que demanda a produção de relatórios padronizados que descrevem o objetivo do modelo, o conjunto de dados empregado, as variáveis de entrada e suas importâncias relativas, além do desempenho segmentado por diferentes grupos populacionais. A literatura propõe abordagens como as Model Cards de Mitchell et al. (2019), que padronizam a documentação de modelos e tornam essas informações comparáveis e transparentes. Incorporar esse tipo de documentação diretamente ao registro do modelo, por exemplo anexando um PDF como artefato, tem se tornado uma prática comum para melhorar a comunicação com stakeholders e facilitar auditorias técnicas.

Outra dimensão fundamental da governança envolve a segregação de ambientes e o controle rigoroso de acesso. Em grandes organizações, diferentes instâncias de Model Registry são mantidas separadamente, como um ambiente de desenvolvimento e outro de produção, complementados por políticas de permissão que restringem quem pode promover, modificar ou excluir modelos. A integração com

mecanismos corporativos de autenticação e autorização, como SSO e RBAC, garante que somente usuários autorizados realizem operações sensíveis. Além disso, é possível automatizar o fluxo de promoção entre ambientes, movendo modelos do registro de desenvolvimento para o produtivo de forma controlada, o que adiciona uma camada extra de segurança e impede que experimentos interfiram inadvertidamente em sistemas em operação.

A governança também incorpora políticas de retenção e arquivamento, especialmente em setores altamente regulados. Certas normas exigem que modelos e seus registros sejam preservados por períodos específicos, como cinco anos após decisões financeiras nas regulamentações bancárias. Para cumprir essas exigências, versões antigas dos modelos são movidas para o estado de arquivadas, permanecendo inacessíveis para uso operacional, mas disponíveis para fins históricos e de compliance. Algumas organizações adotam estratégias de armazenamento de menor custo, como cold storage, após determinado período, assegurando que nenhum modelo seja removido definitivamente sem aprovação formal das áreas jurídicas responsáveis.

De forma geral o Model Registry se alia a práticas organizacionais formais em ambientes mais sensíveis para garantir que a IA da empresa seja confiável, auditável e em conformidade. Cada mudança de modelo torna-se um evento registrado e a organização ganha a habilidade de gerenciar sua frota de modelos da mesma forma profissional e segura com que gerencia seu patrimônio de software tradicional. Não surpreende, portanto, que consultorias e pesquisas recentes apontem para um aumento significativo nos investimentos em ferramentas de governança de IA. Por exemplo, uma análise da Gartner de 2024 indica que as plataformas de AI Governance estão emergindo como componente-chave para garantir escalabilidade e confiança em soluções de IA corporativas. Esse foco crescente em governança reflete a maturidade do campo de ML Engineering: não basta criar modelos eficazes, é preciso garantir que eles continuem eficazes, seguros e éticos ao longo do tempo.

### Integração com ferramentas e ecossistema mlops

Por último, é importante destacar que um Model Registry raramente atua isoladamente – ele costuma ser o núcleo de um ecossistema maior de ferramentas de MLOps. Após o registro e a aprovação, o modelo precisa ser disponibilizado por meio de um serviço de inferência, normalmente exposto como um endpoint REST em produção. Muitas plataformas de Model Registry já oferecem integrações nativas com mecanismos de model serving. O MLflow, por exemplo, disponibiliza o comando `mlflow models serve`, que lê um modelo diretamente do registro e cria rapidamente um servidor de inferência local. Plataformas gerenciadas seguem o mesmo princípio, permitindo o deploy diretamente a partir do registro.

O Amazon SageMaker Model Registry integra-se ao serviço de deployment do próprio SageMaker, enquanto o Azure Machine Learning possibilita implantar uma versão específica do modelo registrado em um Azure Container Instance ou em um cluster Kubernetes com apenas alguns cliques. Essa capacidade reforça o papel do registro como fonte central a partir da qual todos os serviços downstream obtêm os artefatos corretos de modelo, assegurando consistência entre ambientes.

A compatibilidade entre dados de treinamento e produção é igualmente crítica, o que torna os Feature Stores elementos fundamentais nesse ecossistema. Soluções como Feast, Hopsworks e Tecton funcionam como repositórios que armazenam features já preparadas para uso em inferência ou treinamento. O Model Registry pode interagir diretamente com o Feature Store para identificar quais versões de features foram utilizadas em cada ciclo de treinamento. Assim, um modelo de recomendação pode trazer em seus metadados a referência exata da versão das embeddings de produtos utilizadas. Essa integração estabelece uma linhagem de dados completa, facilitando análises de impacto e a identificação de potenciais quedas de desempenho decorrentes de alterações nos dados-base.

Ferramentas de orquestração de pipelines, como Apache Airflow, Google Cloud Composer e AWS Step Functions, também se integram ao Model Registry para automatizar fluxos de MLOps. Uma DAG em Airflow pode treinar um modelo, registrá-lo, consultar a versão atualmente em produção e compará-la com a nova versão com base em uma métrica de negócio. Dependendo do resultado, o pipeline pode notificar

um engenheiro responsável ou até promover automaticamente o novo modelo. Esse tipo de automação compõe abordagens de continual learning ou continuous training, viabilizadas pela integração entre registries e demais tecnologias de MLOps.

Em ambientes de larga escala, equipes de DevOps utilizam sistemas de Application Performance Monitoring como Datadog, New Relic e Dynatrace para coletar métricas e logs. A integração desses sistemas com o Model Registry permite enriquecer métricas operacionais com metadados da versão do modelo em uso. Em um serviço de e-commerce, por exemplo, cada requisição de recomendação pode ser rotulada com o identificador da versão do modelo utilizado, possibilitando correlacionar variações de desempenho ou de KPIs de negócio com a introdução de uma nova versão. Isso acelera diagnósticos e decisões de rollback, reforçando o papel do registry como intermediário essencial que fornece às ferramentas de observabilidade as informações necessárias sobre versões em execução.

Em resumo, o Model Registry se integra ao restante do ecossistema de Engenharia de ML para fornecer governança fim a fim. Desde a ingestão de dados (com feature stores), passando pelo desenvolvimento e treinamento (tracking de experimentos, pipelines CI/CD), até a implantação e monitoramento, o registry permeia todas as fases. Essa abordagem holística é, na prática, o que distingue a Engenharia de Machine Learning moderna (MLOps) do desenvolvimento de modelos “artesanal”: é a diferença entre treinar um modelo uma única vez e torcer para que funcione versus operar dezenas de modelos de forma coordenada, confiável e escalável. Você verá mais sobre essas integrações nas videoaulas, que traz um panorama visual da arquitetura MLOps completa e o papel do Model Registry nela!

## MERCADO, CASE E TENDÊNCIAS

### Cases de sucesso

A ampla adoção de Model Registry e governança de modelos pode ser observada em diversos setores. No setor financeiro, grandes bancos implementam rigorosos processos de governança para modelos de crédito e detecção de fraudes. Por exemplo, o Scotiabank (Canadá) passou a utilizar um registro de modelos integrado ao seu pipeline de fraud detection, permitindo processar milhares de transações por segundo com Machine Learning e reduzindo em cerca de 30% a incidência de falsos positivos nas sinalizações de fraude após a adoção de estratégias de A/B testing controladas pelo registry.

Já no ramo saúde, a HealthTech UnitedHealth Group relatou sucesso ao padronizar um registro de modelos para monitorar algoritmos de diagnóstico: com isso, a empresa consegue rastrear facilmente a origem de recomendações clínicas automatizadas e atender mais rapidamente a auditorias regulatórias. Em e-commerce, o Airbnb desenvolveu o Bighead, plataforma interna que inclui um registry para gerenciar modelos de recomendação e precificação dinâmicos, suportando mais de 1,5 milhão de inferências por segundo e agilizando em 60% o tempo de go-to-production de novos modelos. Na indústria manufatureira, empresas como a Siemens adotam governança de modelos preditivos para manutenção preditiva: cada modelo que monitora máquinas em fábricas é versionado e acompanha relatórios de desempenho, permitindo identificar rapidamente quais versões requerem recalibração ou substituição.

### Publicações e recursos recentes

A pesquisa acadêmica e técnica em MLOps tem dedicado atenção crescente à questão do gerenciamento e governança de modelos. Um exemplo é o [artigo](#) “On Challenges in Machine Learning Model Management” de Schelter et al. (2018), que discute as dificuldades de gerenciar modelos e sugere arquiteturas para solucioná-las. Mais recentemente, Xu et al. (2022) propuseram o Deep Isolation Forest, um

aprimoramento do algoritmo Isolation Forest que combina árvores de decisão com representações de rede neural para melhorar a detecção de anomalias em dados complexos (a técnica alcançou cerca de 15% a mais em AUC em relação ao Isolation Forest tradicional). Em termos de ferramentas, destaca-se a iniciativa de Model Cards mencionada anteriormente (Mitchell et al., 2019) e o lançamento do Microsoft Responsible AI Dashboard (2022), que inclui funcionalidades de governança como documentação automatizada de modelos e verificação de vies.

Outro recurso valioso é o podcast “MLOps Coffee Sessions” (disponível no YouTube e Spotify), no qual profissionais de empresas como Google, Netflix e Uber discutem desafios e soluções práticos de governança de modelos em episódios recentes (2023-2024). Esses materiais complementam a literatura formal e mantêm você atualizado(a) sobre tendências emergentes, como o uso de LLMs (Large Language Models) no gerenciamento de modelos e a incorporação de métricas de negócios diretamente nos pipelines de MLOps.

### **Tendências atuais**

Nos próximos anos, espera-se que a regulamentação de IA se torne mais abrangente, impulsionando ainda mais a necessidade de governança de modelos. A proposta de Lei Europeia de IA (AI Act), por exemplo, deverá entrar em vigor e exigirá que empresas classifiquem seus sistemas de IA por nível de risco e adotem medidas proporcionais de governança, auditoria e segurança. Outra tendência é a convergência de MLOps com gestão de risco e compliance: analistas preveem um aumento na oferta de plataformas integradas de AI Governance que combinam registros de modelo, monitoramento, explicabilidade e gerenciamento de dados sensíveis.

Além disso, soluções de AutoML e aprendizado contínuo começam a incorporar o registro de modelos para treinar e testar centenas de modelos candidatos automaticamente, escolhendo ou combinando os melhores (por exemplo, Google Vertex AI oferece um Model Registry integrado com seu serviço de AutoML). Por fim, impulsionada pelos avanços em IA generativa, a demanda por governança também se estende a modelos de grande porte: empresas como OpenAI e Hugging Face já

disponibilizam ferramentas para gerenciamento de versões e usos permitidos de modelos de linguagem extensos. Em resumo, Model Registry e governança de modelos não são modismos passageiros, mas componentes centrais e em rápida evolução para viabilizar o sucesso sustentado de projetos de Machine Learning na próxima década.

EMANIP



## O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu que construir um modelo acurado é apenas uma parte do desafio – a outra parte crucial é gerenciar esse modelo de forma adequada após o desenvolvimento. Discutimos os problemas que a falta de governança de modelos pode causar, desde erros operacionais até riscos regulatórios, e introduzimos o conceito de Model Registry como solução para centralizar o versionamento, documentação e controle de ciclo de vida dos modelos de Machine Learning. Você viu como o MLflow Model Registry pode ser usado na prática para registrar modelos, criar novas versões automaticamente, promover modelos entre estágios (como Staging e Production) e adicionar metadados de aprovação e descrição para reforçar a governança.

Também exploramos fundamentos teóricos da governança de modelos: definimos políticas de qualidade, conformidade e rastreabilidade que asseguram que apenas modelos confiáveis e auditáveis cheguem à produção. Abordamos como o Model Registry se integra com outros elementos do ecossistema MLOps – de pipelines de CI/CD a monitoramento contínuo –, fechando o ciclo de aprendizado de máquina e permitindo operação escalável e sustentável de modelos ao longo do tempo.

Em suma, você agora compreende que técnicas de governança de modelos e uso de Model Registry são essenciais para o sucesso em projetos de Machine Learning Engineering, garantindo que a inteligência de máquina seja implantada com segurança, eficiência e responsabilidade. Lembre-se de revisar sempre que necessário os conteúdos e exemplos fornecidos – e não hesite em retornar às videoaulas e materiais adicionais para consolidar seu aprendizado e aplicá-lo em seus desafios de MLE!

## REFERÊNCIAS

ALGORITHMIA. **Why Governance Should Drive Your 2021 ML Strategy**. 2021. Disponível em: <https://ai-infrastructure.org/why-governance-should-drive-your-2021-ml-strategy/>. Acesso em: 20 mar. 2026.

GOOGLE CLOUD. **MLOps: Continuous delivery and automation pipelines in machine learning**. 2024. Disponível em: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>. Acesso em: 20 mar. 2026.

MITCHELL, M. et al. **Model Cards for Model Reporting**. Proceedings of FAT\* 2019 – ACM Conference on Fairness, Accountability, and Transparency. Atlanta, GA, 2019.

MLFLOW. **MLflow Model Registry – Documentation (Latest Release)**. 2023. Disponível em: <https://mlflow.org/docs/latest/model-registry.html>. Acesso em: 20 mar. 2026.

SCHELTER, S. et al. **On Challenges in Machine Learning Model Management**. IEEE Data Engineering Bulletin, v.41, n.4, p.5-14, 2018.

SCULLEY, D. et al. **Hidden Technical Debt in Machine Learning Systems**. In: Advances in Neural Information Processing Systems 28 (NIPS 2015). 2015.

STUDER, S. et al. **Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology**. Machine Learning and Knowledge Extraction (MAKE), v.3, n.2, p.392-413, 2021.

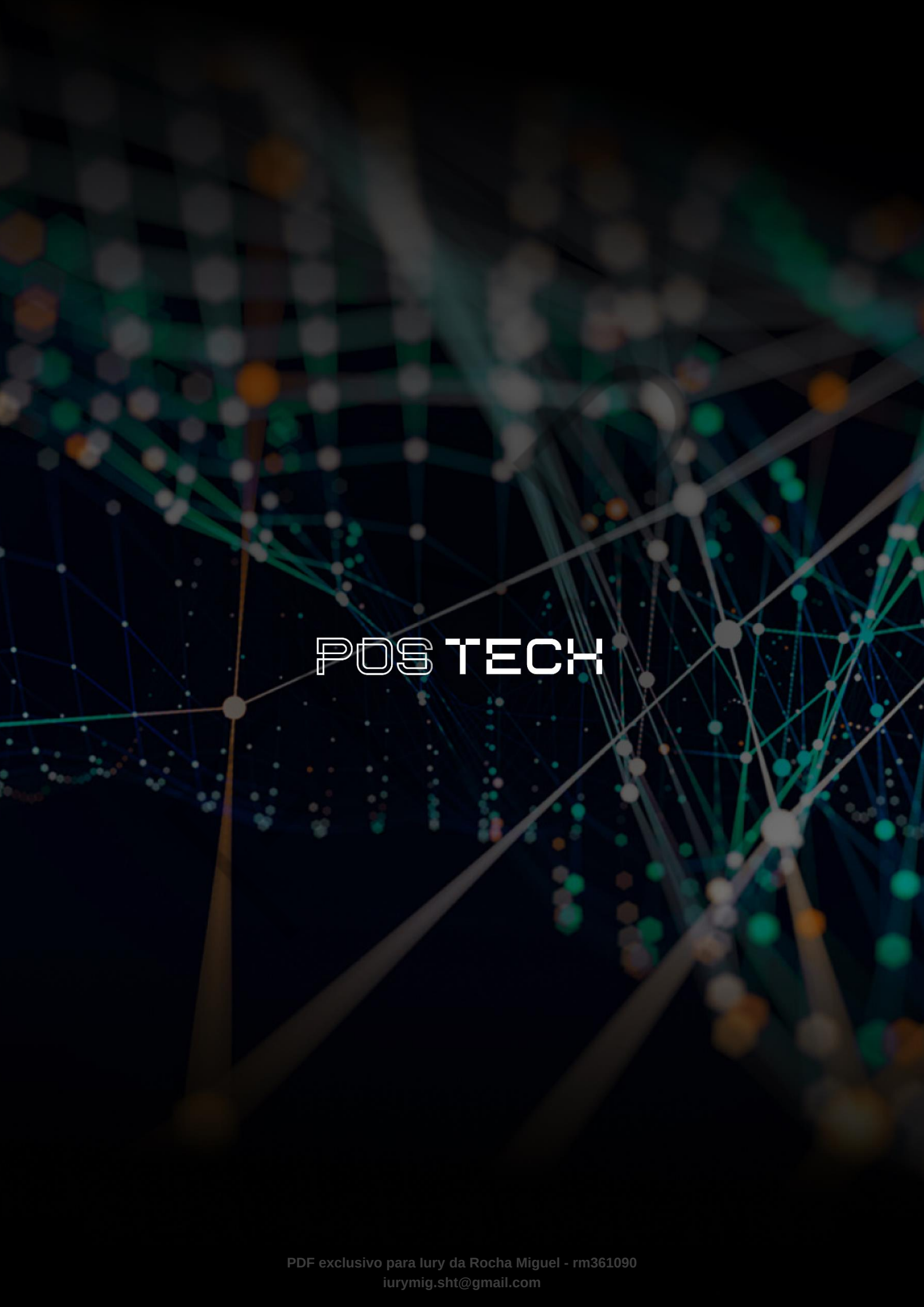
XU, H. et al. **Deep Isolation Forest for Anomaly Detection**. IEEE Trans. on Knowledge and Data Engineering, 2023 (no prelo). Disponível em: <https://arxiv.org/abs/2206.06602>. Acesso em: 20 mar. 2026.

ZAHARIA, M. et al. **Accelerating the Machine Learning Lifecycle with MLflow**. IEEE Data Engineering Bulletin, v.41, n.4, p.39-45, 2018.

## **PALAVRAS-CHAVE**

Model Registry. Governança de Modelos. MLOps.

EMENDAS



# POSTECH